



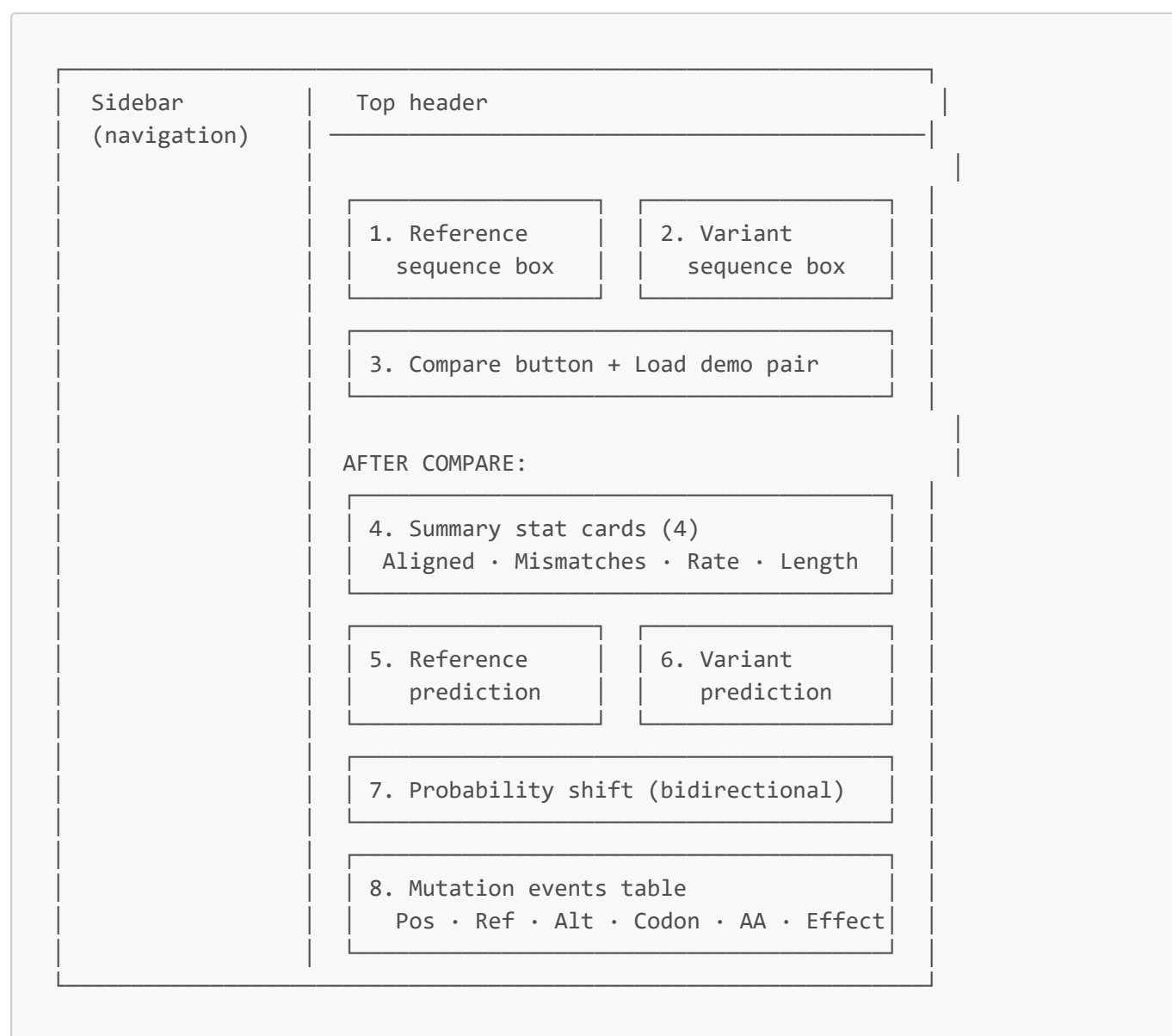
Mutation Lab Page — Complete Component Guide

A deep-dive into every panel and control on the **Mutation Lab** page of GenomelQ. For each component this document covers:

- **What it is** (plain-language description)
- **What it tells you** (how to read the output)
- **How it works under the hood** (the algorithm)
- **How to use it** (the user flow)
- **Practical tips** (when it's useful, when to be cautious)



Page Layout Overview



The Mutation Lab is laid out in two phases: **input** at the top (two side-by-side textareas + a button row) and **results** below (eight cards arranged in a vertical stack with two-column groupings).

1. Reference Sequence Box

What it is

A textarea where you paste the **reference (wild-type)** DNA sequence — the version you consider "normal" for the comparison.

What it tells you

Just an input field. Once compared, this sequence becomes the baseline against which the variant is measured.

How it works under the hood

Bound to `mutation.reference` via Alpine `x-model`. On submit, sent as the `reference` field in `POST /mutation`.

How to use it

1. Paste a full DNA sequence (typically a known gene or wild-type allele).
2. The textarea is monospace and resizable — comfortable for sequences hundreds of bases long.

Practical tips

- **Same character rules as Analyze:** only A/T/G/C/N accepted; case and whitespace stripped.
 - Keep the reference and variant **the same length** for the cleanest comparison. Length mismatches still work but only the overlapping prefix is aligned.
 - Need a starting point? Click **Load demo pair** (#3) to populate both boxes with a one-mutation example.
-

2. Variant Sequence Box

What it is

A second textarea for the **variant (mutated)** sequence — the candidate you want to compare against the reference.

What it tells you

Mirror image of the Reference box. Position-by-position, this sequence will be diffed against the reference.

How it works under the hood

Bound to `mutation.variant` via Alpine `x-model`. Sent as the `variant` field in `POST /mutation`.

How to use it

1. Paste the candidate sequence — usually a single-nucleotide variant of the reference.
2. Or click **Load demo pair** to auto-fill an example.

💡 Practical tips

- For **single-mutation** experiments, copy the reference sequence here, then change exactly one base. The mutation table will pinpoint that change with codon and AA info.
- For **multi-mutation** experiments, every mismatch is reported.
- For **insertion/deletion** experiments, the tool aligns base-by-base — it does NOT do indel-aware alignment. So an insertion at position 50 will appear as a long string of mismatches starting at 50.

3. Compare Button + Load Demo Pair

What it is

Two buttons below the input boxes. The blue **Compare Sequences** triggers the analysis; the gray **Load demo pair** populates both boxes with a ready-to-run example.

What it tells you

Element	Meaning
Compare Sequences	Runs the comparison. Shows a spinner and disables itself while loading.
Load demo pair	Fills the boxes with a BRCA1-derived reference and a one-base variant of it.
Error toast (below)	Shows any validation or backend error.

How it works under the hood

```
runMutation() {
  POST /mutation { reference, variant }
  → mutation.result = response
  → render all 5 result cards
}

mutationSample() {
  this.mutation.reference = SAMPLES.cancer;
  this.mutation.variant   = SAMPLES.cancer.replace(SAMPLES.cancer[50], "C");
}
```

The demo pair changes exactly **one base at position 50** of the BRCA1 sample, so the result will show 1 mismatch and let you see the entire pipeline working end-to-end.

How to use it

1. Either paste your own pair OR click **Load demo pair**.
2. Click **Compare Sequences**.
3. Wait ~50 ms — results render below.

💡 Practical tips

- The demo pair is **always the BRCA1-Cancer sequence** with a single substitution. It's a quick smoke test that the backend, classifier, and rendering all work.
- After comparing, you can **edit either box and re-compare** without clearing — useful for "what if" exploration.

4. Summary Stat Cards

What it is

Four small stat cards displaying high-level mutation comparison numbers.

What it tells you

Card	Meaning
Aligned	Number of positions actually compared. Equals <code>min(len(reference), len(variant))</code> .
Mismatches	Total positions where <code>reference[i] != variant[i]</code> .
Mutation rate	<code>mismatches / aligned</code> , expressed as a percentage.
Length match	Yes (green) if the two sequences have the same length, No (amber) otherwise.

How it works under the hood

Computed in `src/ml/mutation.py` → `compare()`:

```
aligned_len = min(len(ref), len(alt))
events = [position-by-position mismatches]
rate = len(events) / aligned_len
is_length_mismatch = len(ref) != len(alt)
```

How to use it

A glance gives you the **scale** of the comparison:

- 0 mismatches → identical sequences (rate 0%)
- 1 mismatch → single-nucleotide variant (rate ~0.4% for a 250-bp sequence)
- Many mismatches → divergent sequences; check the table below

Practical tips

- A **length mismatch** is a flag that you may have an indel or a partial sequence. The classifier still runs, but the position numbers in the events table only cover the aligned region.
- Mutation rates above ~5% suggest the two sequences aren't really alleles of the same gene — they might be different genes or different organisms.

5. Reference Prediction Card

What it is

A card showing the disease classifier's verdict on the reference sequence.

What it tells you

- **Predicted class** (color-coded by disease)
- **Confidence percentage**

This is what the classifier thinks the **wild-type** version is.

How it works under the hood

`compare()` in `src/ml/mutation.py` calls `predict_sequence(ref)` from `src/ml/classifier.py`. Result is the standard prediction payload (verdict + probabilities), trimmed to the most useful fields for display.

How to use it

Read this card together with #6 (Variant prediction) and #7 (Probability shift) to see how the classifier's view changes between the two sequences.

Practical tips

- If the reference's prediction is itself uncertain (low confidence), the comparison is on shaky ground. Run the reference alone in the Analyze page first to gauge stability.

6. Variant Prediction Card

What it is

A card showing the disease classifier's verdict on the variant sequence.

What it tells you

- **Predicted class** (color-coded)
- **Confidence percentage**

The mirror of #5 — the classifier's view of the **mutated** version.

How it works under the hood

Same as #5 but called on `alt` instead of `ref`.

How to use it

Compare it to #5:

- **Same predicted class, similar confidence** → mutation is essentially benign at the classifier's level (likely synonymous or weakly-impactful missense).

- **Same class, different confidence** → mutation alters confidence but not the verdict.
- **Different class** → the mutation flipped the model's decision; could indicate a high-impact change.

💡 Practical tips

- A class flip is not necessarily clinical evidence — it's a model signal. Always cross-reference with the events table (#8) to see if any mutation is biologically meaningful (nonsense, large effect missense).

7. ↔ Probability Shift (Bidirectional Bars)

🎯 What it is

A card showing, for each disease class, the **change in predicted probability** between reference and variant.

💬 What it tells you

For each class:

- A label (e.g. **Cancer**)
- A horizontal bar centered on a vertical zero-line
- A **green bar to the right** if probability increased (variant more like this class than reference)
- A **red bar to the left** if probability decreased
- A numeric delta on the far right (e.g. **+12.3%**, **-4.1%**)

The deltas always sum to 0 (probabilities sum to 1 on both sides).

⚙️ How it works under the hood

```
ref_probs = predict_sequence(ref).probabilities # {Cancer:0.79, ...}
alt_probs = predict_sequence(alt).probabilities
shifts    = {cls: alt_probs[cls] - ref_probs[cls] for cls in classes}
```

The bar is drawn from the center; width is $\min(|\text{delta}|, 0.5) * 100$ percent, capped at half-width so a 50% shift fills the half-bar fully.

📖 How to use it

Read it as the answer to: *"how did the variant push the model's beliefs?"*

- **A big green bar on Cancer** → the variant looks more cancerous to the model.
- **A big red bar on Normal** with green on Cancer → variant moved the prediction from healthy toward disease.
- **Tiny bars across the board** → the mutation barely registered.

💡 Practical tips

- The threshold for a "real" shift depends on context, but typically anything **> 5%** is worth noting.

- Combine with the **AA effect** column in #8: a missense or nonsense mutation that produces a large probability shift is a high-priority finding.

8. Mutation Events Table

What it is

A detailed, position-by-position list of every mismatch between reference and variant, with biological interpretation.

What it tells you

Each row is one mutation:

Column	Meaning
Pos	1-indexed nucleotide position.
Ref	The reference base (red, monospace).
Alt	The variant base (green, monospace).
Codon	The full 3-base codon containing the mutation (ATG → ATC).
AA	The amino acid translation change (M → I).
Effect	Categorized: synonymous · missense · nonsense · stop_loss · unknown .

The Effect column is **color-badge-coded** for fast scanning:

-  **synonymous** (no AA change) → green
-  **missense** (different AA) → amber
-  **nonsense** (creates a stop codon) → red
-  **stop_loss** (a stop codon mutated to a sense codon) → purple

How it works under the hood

`src/ml/mutation.py` → `compare()` walks position by position:

```
for i in range(aligned_len):
    if ref[i] != alt[i]:
        codon_idx = i // 3
        ref_codon = ref[codon_idx*3 : codon_idx*3 + 3]
        alt_codon = alt[codon_idx*3 : codon_idx*3 + 3]
        ref_aa = Bio.Seq.Seq(ref_codon).translate()
        alt_aa = Bio.Seq.Seq(alt_codon).translate()
        effect = classify(ref_aa, alt_aa) # synonymous / missense / nonsense /
stop_loss
        events.append(MutationEvent(...))
```

Up to 200 events are returned per request to avoid pathological diffs from blowing up the response.

How to use it

For each row:

1. Note the **position** to know where in the sequence.
2. Look at the **AA** column. **M** → **I** is a missense; **Q** → ***** is a nonsense.
3. Check the **Effect** badge for the categorical label.
4. If you want context, the **codon** column shows exactly which 3 bases were affected.

Practical tips

- The table caps display at **40 rows** in the UI to keep the page snappy. The full list (up to 200 events) is in the JSON response — query the API directly for unlimited.
- **Synonymous mutations rarely cause disease** but can affect splicing or expression — don't dismiss them outright.
- **Nonsense mutations are usually high-impact** because they truncate the protein.
- The codon framing assumes the sequence **starts at frame 0**. If your sequence starts mid-gene, the codon/AA columns will be off-by-one or off-by-two — interpret accordingly.

The Full Mutation Lab Flow (End-to-End)

```

User pastes reference + variant
    ↓
clicks Compare Sequences
    ↓
[POST /mutation] { reference, variant }
    ↓
Backend: src/api/service.py → mutation_payload()
    |
    ├── validate_sequence(reference) → reject invalid
    ├── validate_sequence(variant)
    |
    └── src/ml/mutation.py → compare()
        |
        ├── align position-by-position (no indel handling)
        ├── for each mismatch:
        |   ├── extract codon (i // 3)
        |   ├── translate to AA via Biopython
        |   └── classify effect
        |
        ├── predict_sequence(reference) → ref_probs
        ├── predict_sequence(variant) → alt_probs
        |
        └── return MutationReport {
            aligned_length, total_mismatches,
            mutation_rate, is_length_mismatch,
            events: list[MutationEvent],
            ref_prediction, alt_prediction,
            probability_shifts
        }
  
```



```
↓  
JSON returned to UI  
↓  
Alpine state.mutation.result is set  
↓  
All 5 result cards render reactively
```

Total time: ~**100 ms** for typical inputs (one or a few mismatches).

Cheat Sheet — Interpreting Results

Pattern	Likely meaning
0 mismatches	Sequences identical — comparison is a sanity check
1 mismatch + synonymous	Silent variant — probably benign at AA level
1 mismatch + missense + small prob shift	Conservative substitution — biologically plausible variant
1 mismatch + missense + large prob shift	High-impact substitution worth follow-up
1 mismatch + nonsense	Premature stop codon — protein truncated, often pathogenic
Many mismatches + class flip	Sequences too divergent to be alleles
Length mismatch + many trailing mismatches	Likely an indel — tool can't align properly

Related Documents

- [README.md](#) — high-level project overview
- [DOCS.md](#) — comprehensive technical documentation
- [HOW_TO_RUN_IN_VSCODE.md](#) — VS Code setup
- [TESTING_GUIDE.md](#) — full test plan including mutation cases
- [Guide/ANALYZE_PAGE_GUIDE.pdf](#) — predictor walkthrough (predictions reused here)

Part of the GenomeIQ per-page documentation series.